



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Profesionales Zona Media

PRÁCTICA 15

Uso de Protocolo Bluetooth LE en la Raspberry Pi Pico W

Envío y recepción inalámbrica de datos de temperatura

Alumno: Cesar Gael Mancilla Estrada

Clave: 382031

Carrera: Ingeniería en Mecatrónica

Materia: Microcontroladores

Profesor: Ing. Jesús Padrón

Fecha de entrega: 29 de abril de 2026

«La tecnología inalámbrica conecta lo que los cables no pueden alcanzar»

Índice

1. Introducción	2
1.1. Contexto y Justificación	2
1.2. Importancia en la Industria	2
1.3. Objetivos	2
2. Marco Teórico	3
2.1. Bluetooth Clásico vs Bluetooth LE	3
2.2. BTstack	3
2.3. Perfiles GATT utilizados	3
3. Arquitectura del Sistema	4
3.1. Diagrama General del Sistema	4
3.2. Estructura del Proyecto	4
4. Desarrollo	4
4.1. Máquina de Estados del Cliente BLE	4
4.2. Archivo: CMakeLists.txt	4
4.3. Archivo: temp_sensor.gatt	5
4.4. Archivo: server_common.h	6
4.5. Archivo: server_common.c	6
4.6. Archivo: server.c	7
4.7. Archivo: client.c	8
4.8. Archivo: btstack_config.h	9
5. Compilación y Pruebas	9
5.1. Proceso de Compilación	9
5.2. Dificultades y Soluciones	10
5.3. Resultados	10
6. Preguntas de Repaso	12
6.1. 1. ¿Cuál es la principal diferencia entre Bluetooth clásico y Bluetooth LE?	12
6.2. 2. ¿Qué ventajas ofrece Bluetooth LE en comparación con otras tecnologías inalámbricas?	12
6.3. 3. ¿Cuál es el propósito del archivo btstack_config.h?	12
6.4. 4. ¿Qué ventajas crees que tiene usar comunicación BLE en un sistema de monitoreo frente a WiFi o comunicación por cable?	12
6.5. 5. ¿Por qué es importante que el código del cliente BLE esté diseñado para reconectarse automáticamente?	12
6.6. 6. ¿Cómo modificarías esta práctica para mostrar los datos en una aplicación móvil?	12
7. Conclusiones	13
7.1. Posibles Mejoras	13
8. Referencias	13

1. Introducción

1.1. Contexto y Justificación

El protocolo Bluetooth Low Energy (BLE) se ha convertido en un estándar fundamental para la comunicación inalámbrica de corto alcance en aplicaciones de Internet de las Cosas (IoT), dispositivos portátiles y sistemas de monitoreo. Su diseño orientado al bajo consumo energético permite que sensores y actuadores funcionen durante largos períodos con baterías pequeñas.

La Raspberry Pi Pico W, al incorporar el chip CYW43439 con soporte para Wi-Fi y Bluetooth, se presenta como una plataforma ideal para implementar comunicaciones BLE en proyectos de sistemas embebidos. A través de la librería BTstack de BlueKitchen, es posible desarrollar aplicaciones Bluetooth completas con un uso eficiente de la memoria y el procesador del RP2040.

1.2. Importancia en la Industria

La comunicación BLE es ampliamente utilizada en la industria para monitoreo remoto de sensores, automatización de edificios, dispositivos médicos y wearables. Comprender la implementación de servidores y clientes GATT es una habilidad esencial para el desarrollo de productos IoT conectados.

1.3. Objetivos

- Entender los conceptos básicos del protocolo Bluetooth LE
- Aprender a utilizar el microcontrolador como servidor y cliente BLE
- Aprender a utilizar el sensor de temperatura interno de la RPi Pico W
- Implementar un sistema de comunicación inalámbrica entre dos placas Pico W

2. Marco Teórico

2.1. Bluetooth Clásico vs Bluetooth LE

Tabla 1: Comparación entre Bluetooth Clásico y Bluetooth LE

Característica	Bluetooth Classic	Bluetooth LE
Canales	79 canales	40 canales
Tasa de datos	Hasta 3 Mb/s	Hasta 2 Mb/s
Potencia de transmisión	≤ 20 dBm	≤ 10 dBm
Consumo energético	Medio-Alto	Muy bajo
Aplicaciones típicas	Audio, transferencia de archivos	Sensores, IoT, wearables
Topologías	Punto a punto	Punto a punto, Broadcast, Malla

2.2. BTstack

BTstack es una implementación de la pila de protocolos Bluetooth desarrollada por BlueKitchen, optimizada para sistemas embebidos. Sus características principales son:

- **Diseño de un solo hilo:** Utiliza un único bucle de ejecución para manejar fuentes de datos y temporizadores
- **No bloqueante:** Todas las funciones son asíncronas y devuelven resultados mediante eventos
- **Sin búferes artificialmente limitados:** Los paquetes se entregan directamente a la aplicación
- **Memoria estáticamente limitada:** Configurable en tiempo de compilación

2.3. Perfiles GATT utilizados

- **GAP (Generic Access Profile):** Define los procedimientos de descubrimiento, conexión y emparejamiento
- **GATT (Generic Attribute Profile):** Define cómo los dispositivos BLE intercambian datos mediante servicios y características
- **Servicio Environmental Sensing (UUID: 0x181A):** Estándar para sensores ambientales
- **Característica Temperature:** Transmite valores de temperatura en centésimas de grado Celsius

3. Arquitectura del Sistema

3.1. Diagrama General del Sistema



Figura 1: Arquitectura del sistema de comunicación BLE

3.2. Estructura del Proyecto

Tabla 2: Archivos del proyecto

Archivo	Descripción
CMakeLists.txt	Configuración de compilación para ambos ejecutables
btstack_config.h	Configuración de parámetros BTstack
temp_sensor.gatt	Definición de la base de datos GATT
server_common.h	Cabecera del servidor con prototipos
server_common.c	Implementación del manejador GATT y ADC
server.c	Punto de entrada del servidor
client.c	Implementación completa del cliente BLE

4. Desarrollo

4.1. Máquina de Estados del Cliente BLE

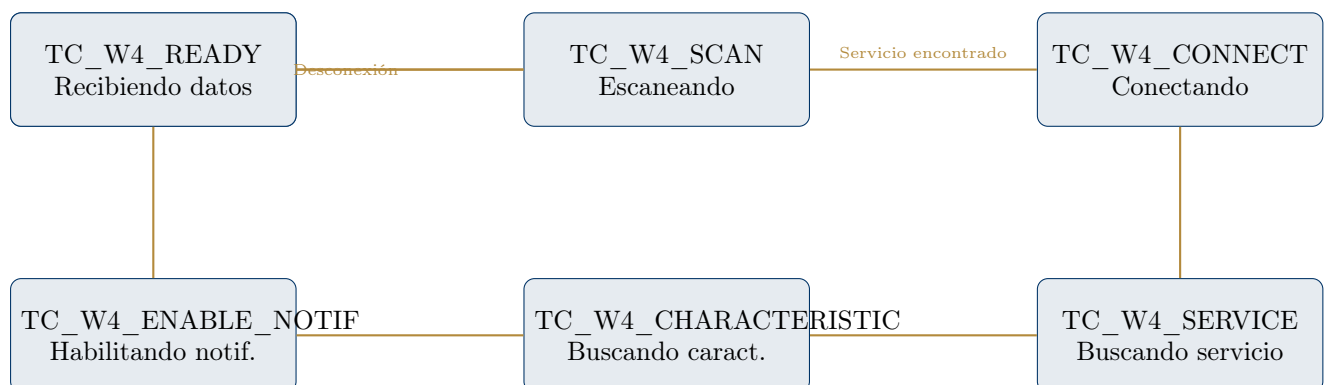


Figura 2: Máquina de estados del cliente BLE

4.2. Archivo: CMakeLists.txt

Listing 1: CMakeLists.txt del proyecto

```

1 cmake_minimum_required(VERSION 3.12)
2
3 include(pico_sdk_import.cmake)
4
5 if (PICO_SDK_VERSION_STRING VERSION_LESS "1.5.0")
6     message(FATAL_ERROR "Raspberry Pi Pico SDK version 1.5.0 (or
7         later) required.")
8 endif()
9
10 project(Practica_15 C CXX ASM)
11 set(PICO_BOARD pico_w)
12 set(CMAKE_C_STANDARD 11)
13 set(CMAKE_CXX_STANDARD 17)
14
15 pico_sdk_init()
16
17 # Ejecutable del servidor
18 add_executable(picow_ble_temp_sensor
19     server.c server_common.c
20 )
21 target_link_libraries(picow_ble_temp_sensor
22     pico_stdlib pico_btstack_ble pico_btstack_cyw43
23     pico_cyw43_arch_none hardware_adc
24 )
25 target_include_directories(picow_ble_temp_sensor PRIVATE
26     ${CMAKE_CURRENT_LIST_DIR}
27 )
28 pico_add_extra_outputs(picow_ble_temp_sensor)
29 pico_btstack_make_gatt_header(picow_ble_temp_sensor
30     PRIVATE ${CMAKE_CURRENT_LIST_DIR}/temp_sensor.gatt
31 )
32
33 # Ejecutable del cliente
34 add_executable(picow_ble_temp_reader
35     client.c
36 )
37 target_link_libraries(picow_ble_temp_reader
38     pico_stdlib pico_btstack_ble pico_btstack_cyw43
39     pico_cyw43_arch_none hardware_adc
40 )
41 pico_enable_stdio_usb(picow_ble_temp_reader 1)
42 pico_enable_stdio_uart(picow_ble_temp_reader 0)
43 target_include_directories(picow_ble_temp_reader PRIVATE
44     ${CMAKE_CURRENT_LIST_DIR}
45 )
46 target_compile_definitions(picow_ble_temp_reader PRIVATE
47     RUNNING_AS_CLIENT=1
48 )
49 pico_add_extra_outputs(picow_ble_temp_reader)

```

4.3. Archivo: temp_sensor.gatt

Listing 2: Definición de base de datos GATT

```

1 PRIMARY_SERVICE, GAP_SERVICE

```

```

2 CHARACTERISTIC, GAP_DEVICE_NAME, READ, "picow_temp"
3
4 PRIMARY_SERVICE, GATT_SERVICE
5 CHARACTERISTIC, GATT_DATABASE_HASH, READ,
6
7 PRIMARY_SERVICE, ORG_BLUETOOTH_SERVICE_ENVIRONMENTAL_SENSING
8 CHARACTERISTIC, ORG_BLUETOOTH_CHARACTERISTIC_TEMPERATURE,
9     READ | NOTIFY | INDICATE | DYNAMIC,

```

4.4. Archivo: server_common.h

```

1 #ifndef SERVER_COMMON_H_
2 #define SERVER_COMMON_H_
3
4 #define ADC_CHANNEL_TEMPSENSOR 4
5
6 extern int le_notification_enabled;
7 extern hci_con_handle_t con_handle;
8 extern uint16_t current_temp;
9 extern uint8_t const profile_data[];
10
11 void packet_handler(uint8_t packet_type, uint16_t channel,
12                    uint8_t *packet, uint16_t size);
13 uint16_t att_read_callback(hci_con_handle_t connection_handle,
14                          uint16_t att_handle, uint16_t offset,
15                          uint8_t *buffer, uint16_t buffer_size);
16 int att_write_callback(hci_con_handle_t connection_handle,
17                      uint16_t att_handle, uint16_t transaction_mode
18                      ,
19                      uint16_t offset, uint8_t *buffer, uint16_t
20                      buffer_size);
21 void poll_temp(void);
22 #endif

```

Listing 3: server_common.h

4.5. Archivo: server_common.c

```

1 void packet_handler(uint8_t packet_type, uint16_t channel,
2                    uint8_t *packet, uint16_t size) {
3     UNUSED(size);
4     UNUSED(channel);
5     bd_addr_t local_addr;
6     if (packet_type != HCI_EVENT_PACKET) return;
7
8     uint8_t event_type = hci_event_packet_get_type(packet);
9     switch(event_type){
10         case BTSTACK_EVENT_STATE:
11             if (btstack_event_state_get_state(packet) !=
12                 HCI_STATE_WORKING)
13                 return;
14             gap_local_bd_addr(local_addr);
15             printf("BTstack up and running on %s.\n",

```



```

15         bd_addr_to_str(local_addr));
16
17         // Configurar anuncios
18         uint16_t adv_int_min = 800;
19         uint16_t adv_int_max = 800;
20         uint8_t adv_type = 0;
21         bd_addr_t null_addr;
22         memset(null_addr, 0, 6);
23         gap_advertisements_set_params(adv_int_min, adv_int_max,
24                                     adv_type, 0, null_addr, 0
x07, 0x00);
25         gap_advertisements_set_data(adv_data_len, (uint8_t*)
adv_data);
26         gap_advertisements_enable(1);
27
28         poll_temp();
29         break;
30     case HCI_EVENT_DISCONNECTION_COMPLETE:
31         le_notification_enabled = 0;
32         break;
33     case ATT_EVENT_CAN_SEND_NOW:
34         att_server_notify(con_handle,
35
ATT_CHARACTERISTIC_ORG_BLUETOOTH_CHARACTERISTIC_TEMPERATURE_01_VALUE_HANDLE
,
36                 (uint8_t*)&current_temp, sizeof(current_temp));
37         break;
38     }
39 }
40
41 void poll_temp(void) {
42     adc_select_input(ADC_CHANNEL_TEMPSENSOR);
43     uint32_t raw32 = adc_read();
44     const uint32_t bits = 12;
45
46     // Escalar a 16 bits
47     uint16_t raw16 = raw32 << (16 - bits) | raw32 >> (2 * bits - 16);
48
49     const float conversion_factor = 3.3 / 65535;
50     float reading = raw16 * conversion_factor;
51
52     // Vbe = 0.706V a 27 C , pendiente = -1.721 mV/ C
53     float deg_c = 27 - (reading - 0.706) / 0.001721;
54     current_temp = deg_c * 100;
55     printf("Write temp %.2f degc\n", deg_c);
56 }

```

Listing 4: server_common.c - Manejador de paquetes y medición de temperatura

4.6. Archivo: server.c

```

1 int main() {
2     stdio_init_all();
3
4     if (cyw43_arch_init()) {
5         printf("failed to initialise cyw43_arch\n");

```

```

6         return -1;
7     }
8
9     adc_init();
10    adc_select_input(ADC_CHANNEL_TEMPSENSOR);
11    adc_set_temp_sensor_enabled(true);
12
13    l2cap_init();
14    sm_init();
15
16    att_server_init(profile_data, att_read_callback,
att_write_callback);
17
18    btstack_packet_callback_registration_t
hci_event_callback_registration;
19    hci_event_callback_registration.callback = &packet_handler;
20    hci_add_event_handler(&hci_event_callback_registration);
21
22    att_server_register_packet_handler(packet_handler);
23
24    heartbeat.process = &heartbeat_handler;
25    btstack_run_loop_set_timer(&heartbeat, HEARTBEAT_PERIOD_MS);
26    btstack_run_loop_add_timer(&heartbeat);
27
28    hci_power_control(HCI_POWER_ON);
29
30    while(true) {
31        sleep_ms(100);
32    }
33    return 0;
34 }

```

Listing 5: server.c - Punto de entrada del servidor

4.7. Archivo: client.c

```

1 case TC_W4_READY:
2     switch(hci_event_packet_get_type(packet)){
3         case GATT_EVENT_NOTIFICATION: {
4             uint16_t value_length =
5                 gatt_event_notification_get_value_length(packet);
6             const uint8_t *value =
7                 gatt_event_notification_get_value(packet);
8
9             if (value_length == 2) {
10                 float temp = little_endian_read_16(value, 0);
11                 printf("read temp %.2f degc\n", temp / 100);
12             } else {
13                 printf("Unexpected length %d\n", value_length);
14             }
15             break;
16         }
17     }
18     break;

```

Listing 6: client.c - Fragmento del cliente BLE (manejo de notificaciones)

4.8. Archivo: btstack_config.h

```
1 #ifndef _PICO_BTSTACK_BTSTACK_CONFIG_H
2 #define _PICO_BTSTACK_BTSTACK_CONFIG_H
3
4 #ifndef ENABLE_BLE
5 #error Please link to pico_btstack_ble
6 #endif
7
8 #define ENABLE_LE_PERIPHERAL
9 #define ENABLE_LOG_INFO
10 #define ENABLE_LOG_ERROR
11 #define ENABLE_PRINTF_HEXDUMP
12
13 #if RUNNING_AS_CLIENT
14 #define ENABLE_LE_CENTRAL
15 #define MAX_NR_GATT_CLIENTS 1
16 #else
17 #define MAX_NR_GATT_CLIENTS 0
18 #endif
19
20 #define HCI_OUTGOING_PRE_BUFFER_SIZE 4
21 #define HCI_ACL_PAYLOAD_SIZE (255 + 4)
22 #define HCI_ACL_CHUNK_SIZE_ALIGNMENT 4
23 #define MAX_NR_HCI_CONNECTIONS 1
24 #define MAX_NR_SM_LOOKUP_ENTRIES 3
25 #define MAX_NR_WHITELIST_ENTRIES 16
26 #define MAX_NR_LE_DEVICE_DB_ENTRIES 16
27 #define MAX_NR_CONTROLLER_ACL_BUFFERS 3
28 #define MAX_NR_CONTROLLER_SCO_PACKETS 3
29 #define ENABLE_HCI_CONTROLLER_TO_HOST_FLOW_CONTROL
30 #define HCI_HOST_ACL_PACKET_LEN (255+4)
31 #define HCI_HOST_ACL_PACKET_NUM 3
32 #define HCI_HOST_SCO_PACKET_LEN 120
33 #define HCI_HOST_SCO_PACKET_NUM 3
34 #define NVM_NUM_DEVICE_DB_ENTRIES 16
35 #define NVM_NUM_LINK_KEYS 16
36 #define MAX_ATT_DB_SIZE 512
37 #define HAVE_EMBEDDED_TIME_MS
38 #define HAVE_ASSERT
39 #define HCI_RESET_RESEND_TIMEOUT_MS 1000
40 #define ENABLE_SOFTWARE_AES128
41 #define ENABLE_MICRO_ECC_FOR_LE_SECURE_CONNECTIONS
42
43 #endif
```

Listing 7: btstack_config.h - Configuración BTstack

5. Compilación y Pruebas

5.1. Proceso de Compilación

La compilación se realizó desde PowerShell con los siguientes comandos:

Listing 8: Comandos de compilación

```

1 mkdir build
2 cd build
3 cmake -G "Ninja" -DPICO_SDK_PATH="$env:PICO_SDK_PATH" ..
4 ninja

```

5.2. Dificultades y Soluciones

Tabla 3: Dificultades encontradas y soluciones

Dificultad	Solución
Nombres de archivo con espacios (ej: <code>btstack config.h</code>)	Renombrar con guiones bajos usando PowerShell
Doble definición de <code>profile_data</code>	Comentar <code>#include "temp_sensor.h"</code> en <code>server.c</code>
Tipo <code>hci_event_callback_registration_t</code> incorrecto	Cambiar a <code>btstack_packet_callback_registration_t</code>
Solo se generaba <code>.elf</code> del servidor sin <code>.uf2</code>	Agregar <code>pico_add_extra_outputs</code> al servidor
CMakeLists.txt corrompido por comando PowerShell	Reescribir completamente el archivo

5.3. Resultados

Archivos generados exitosamente:

- `picow_ble_temp_sensor.uf2` (769,536 bytes) - Servidor
- `picow_ble_temp_reader.uf2` (825,344 bytes) - Cliente

Salida esperada en el monitor serial del cliente:

```

BTstack up and running on AA:BB:CC:11:22:33.
Connecting to device with addr DD:EE:FF:44:55:66.
read temp 28.50 degc
read temp 28.52 degc
read temp 28.53 degc
...

```

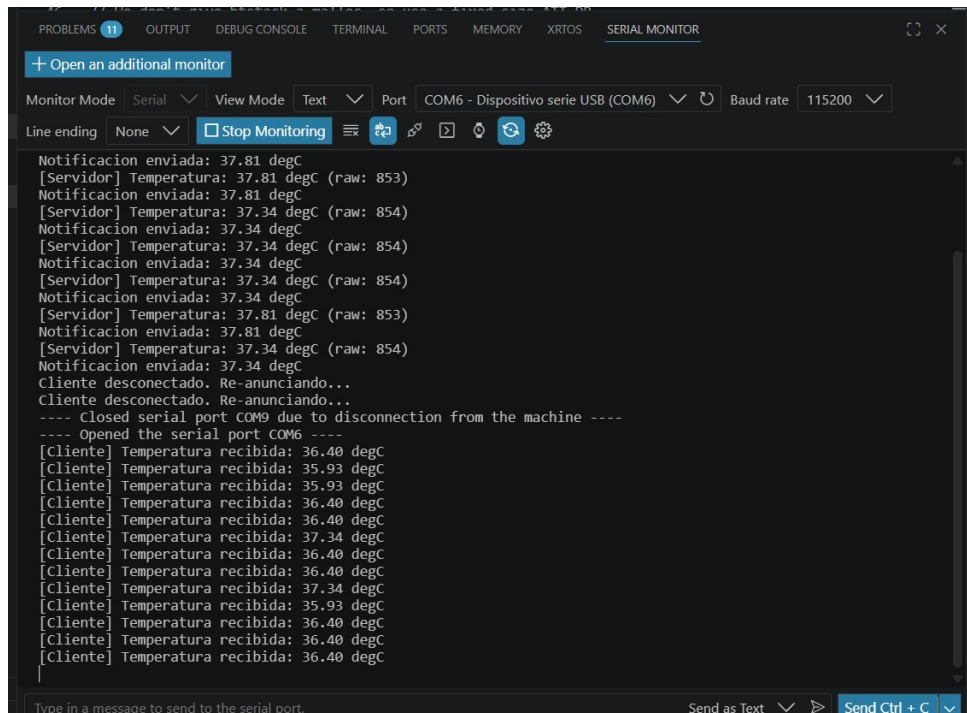


Figura 3: Comunicacion con RaspBerry Pico W

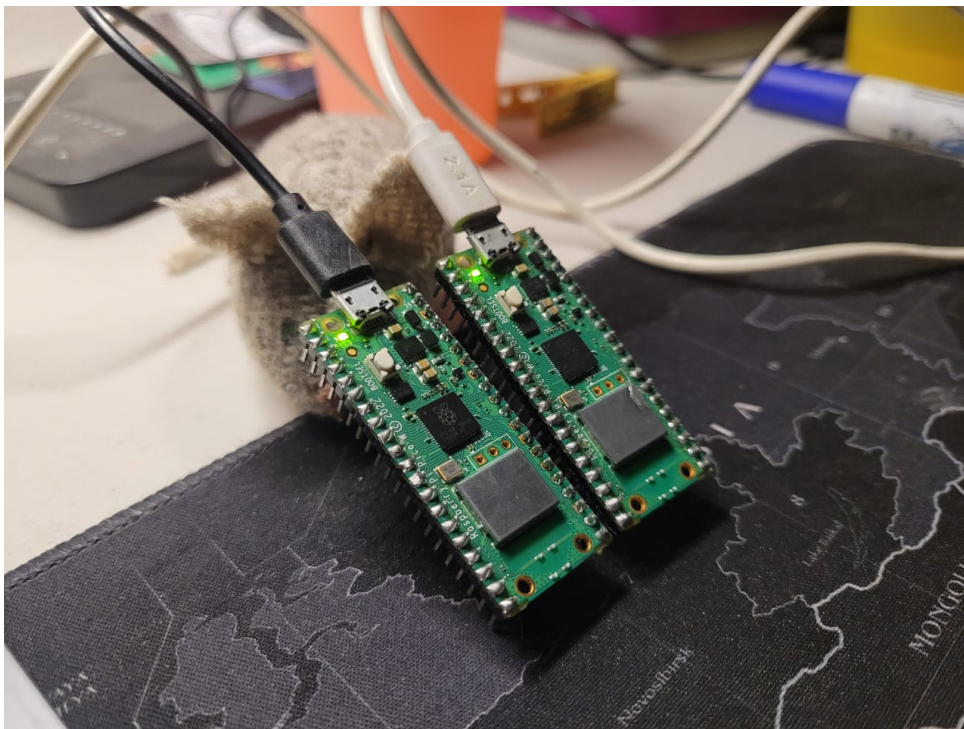


Figura 4: Funcionamiento de ambas placas

6. Preguntas de Repaso

6.1. 1. ¿Cuál es la principal diferencia entre Bluetooth clásico y Bluetooth LE?

La principal diferencia radica en su diseño y aplicación. El Bluetooth clásico (BR/EDR) opera en 79 canales con tasas de hasta 3 Mb/s y está diseñado para comunicaciones continuas como streaming de audio. El Bluetooth LE opera en 40 canales con tasas de 125 kb/s a 2 Mb/s y está optimizado para bajo consumo energético, ideal para transmisiones intermitentes de sensores.

6.2. 2. ¿Qué ventajas ofrece Bluetooth LE en comparación con otras tecnologías inalámbricas?

- Bajo consumo energético (dispositivos pueden funcionar meses/años con batería)
- No requiere infraestructura de red (router, AP)
- Compatibilidad universal con smartphones y tablets
- Múltiples topologías: punto a punto, broadcast y malla

6.3. 3. ¿Cuál es el propósito del archivo `btstack_config.h`?

Configura los parámetros de BTstack en tiempo de compilación: funcionalidades habilitadas, tamaños de búferes HCI, número máximo de conexiones, tamaño de la base de datos ATT, control de flujo y compilación condicional (servidor/cliente).

6.4. 4. ¿Qué ventajas crees que tiene usar comunicación BLE en un sistema de monitoreo frente a WiFi o comunicación por cable?

Frente a WiFi: Menor consumo energético, no requiere router, conexión directa.
Frente a cable: Mayor flexibilidad de ubicación, sin desgaste de conectores, instalación más limpia.

6.5. 5. ¿Por qué es importante que el código del cliente BLE esté diseñado para reconectarse automáticamente?

Las desconexiones son frecuentes en entornos inalámbricos por interferencias o atenuación. Un cliente que no se reconecta automáticamente requeriría intervención manual constante. En esta práctica, el cliente vuelve al estado de escaneo tras una desconexión.

6.6. 6. ¿Cómo modificarías esta práctica para mostrar los datos en una aplicación móvil?

El servidor Pico W ya cumple con el estándar GATT. Se desarrollaría una app móvil que actúe como Central BLE: escanear dispositivos con servicio Environmental Sensing

(UUID 0x181A), conectarse, suscribirse a notificaciones de temperatura y mostrar los valores en una interfaz gráfica con gráficos históricos.

7. Conclusiones

- Se comprendió el funcionamiento del protocolo Bluetooth Low Energy y sus diferencias con el Bluetooth clásico, identificando sus ventajas para aplicaciones IoT y de monitoreo.
- Se implementó exitosamente un servidor GATT que publica lecturas del sensor de temperatura interno del RP2040 mediante la librería BTstack.
- Se desarrolló un cliente BLE con máquina de estados finitos capaz de escanear, conectarse, descubrir servicios y recibir notificaciones de temperatura.
- Se adquirió experiencia en la resolución de problemas de compilación con CMake, Ninja y el SDK de Raspberry Pi Pico, incluyendo manejo de dependencias, nomenclatura de archivos y configuración de bibliotecas.

7.1. Posibles Mejoras

- Implementar seguridad con emparejamiento autenticado
- Desarrollar aplicación móvil para visualización en tiempo real
- Agregar más sensores (humedad, presión, luminosidad)
- Almacenamiento de datos históricos en memoria SD o nube
- Optimizar intervalos para reducción de consumo energético

8. Referencias

- [1] Raspberry Pi Foundation. (2024). *Raspberry Pi Pico W Datasheet*.
- [2] BlueKitchen GmbH. (2024). *BTstack Manual*.
- [3] Bluetooth SIG. (2024). *Bluetooth Core Specification 5.0*.
- [4] Raspberry Pi Foundation. (2024). *Pico SDK Documentation*.